

3.8 - Rate limiting

Proteger todos contra o excesso de um

Um bug de terceiro derruba a plataforma

- **Transportadora integrou a API com um bug grave**
 - Aplicação dela dispara oitocentas requisições por segundo

LUNALOG

Uma transportadora integrou a API da LunaLog de forma incorreta, e um bug no código dela fazia a aplicação disparar oitocentas requisições por segundo contra um único endpoint. Em menos de um minuto, o banco de dados estava a cem por cento de CPU e a plataforma começou a rejeitar requisições de todos os outros clientes. O problema não era má-fé, era um simples bug. Mas sem limite de taxa por cliente, o erro de um terceiro derrubou o sistema inteiro, e ficou claro que confiar na boa vontade dos integradores nunca foi uma estratégia.

Algoritmos de rate limiting

Como contar e barrar o excesso

- **Fixed window conta requisições em janelas fixas**
 - Simples, mas permite rajada na virada da janela
- **Sliding window suaviza a contagem ao longo do tempo**
 - Evita o pico de borda do fixed window
- **Token bucket permite rajadas controladas**
 - Tokens acumulam e são gastos a cada requisição
- **Leaky bucket força vazão constante de saída**
 - Excesso transborda e é descartado ou enfileirado

Permitir rajada ou forçar ritmo

Token bucket

- Permite rajadas curtas dentro do orçamento
- Bom para tráfego naturalmente irregular
- Tokens acumulados absorvem picos legítimos
- Configura taxa média e tamanho do burst

Leaky bucket

- Impõe vazão de saída perfeitamente constante
- Bom para proteger recurso sensível a pico
- Excesso é descartado ou aguarda na fila
- Suaviza o tráfego antes de chegar ao serviço

Comunicar o limite, não só aplicá-lo

- Limite por cliente isola o abuso de um só integrador
- Cliente bem informado se adapta sozinho ao limite

Devolva headers padronizados como `RateLimit-Limit`, `RateLimit-Remaining` e `Retry-After`. Um 429 com `Retry-After` ensina o cliente a recuar, em vez de insistir e piorar.



Rate limiting protege o seu servidor dos clientes. Mas existe a direção oposta do problema: quem protege o seu serviço quando uma dependência que ele chama começa a falhar ou a responder devagar? Sem proteção, a lentidão de um sistema externo vira o colapso interno do seu, em questão de segundos.

Circuit breaker e bulkhead: conter a cascata